E.M.B.E.R (EMergency Building Entry Robot)

Group 33 | | Hunor Adam Kovacs (6549608) | Sam Leveau (6374670) | Kai Henriquez (5480019) | Enan Bester (6534899)

***Abstract*— This project develops and implements an autonomous quadcopter for indoor fire surveillance navigation using global sampling-based path planning and local Model Predictive Control (MPC). Collision-free paths generated by RRT, RRT*, RRT-Connect, and BIT* are compared and tracked with MPC in static environments, with exploration of dynamic obstacle handling.**

I. INTRODUCTION

Traditional structural surveillance firefighting methods rely heavily on human interaction. These tasks put firefighters in significant danger as they enter burning or structurally unstable buildings while scouting for victims or assessing fire spread. Additionally, these techniques are slow to gather and distribute important information.

As a first step toward addressing this problem, this project aims to develop a motion-planning quadcopter capable of navigating into and through a building floor towards a set exit. This serves as an exploration of how autonomous systems can further support safer fire surveillance.

A. Relevant work

Autonomous quadcopters have been widely developed and studied for dangerous field operations such as search and rescue, outdoor inspection, and disaster response [3]. In contrast, there is comparatively less research focused on indoor surveillance, where confined spaces and the absence of GPS complicate autonomous flight. Instead, indoor research has emphasized high-speed autonomous navigation and obstacle avoidance, as surveyed in *A Survey of Indoor UAV Obstacle Avoidance Research* [4]. A common control method utilized in these environments is Model Predictive Control (MPC), which enables predictive constraint-based motion planning.

Prior research commonly applies MPC to linear or linearized quadcopter models in order to enable real-time implementation, rather than solving a fully nonlinear optimization problem. For example, in *Real-Time Model Predictive Control for Quadrotors*, the nonlinear dynamics were reduced via feedback linearization before applying MPC [5]. This project follows the same approach by applying MPC on a simplified hovering linear dynamic model.

Relevant research on global path planning in indoor environments typically relies on planners that use a discrete graph representation of the environment, to avoid the need for explicit collision checking [6]. This would not suffice for our application as a prediscretized space would not be possible and due to tradeoffs in those spaces that either miss narrow passages or become computationally costly. Therefore the decision was made to proceeded with a multitude of sampling based planners such as RRT, RRT*, RRT-connect [2] and BIT* [1].

Author contributions[1] and use of generative AI [2].

II. PROBLEM DEFINITION

Given a fully known indoor environment with static and/or dynamic obstacles, the objective is to autonomously navigate a quadcopter from a start position to a goal position while avoiding collisions. The problem is formulated as a motion planning task, where a collision-free trajectory must be generated by a global planner. This path is then tracked by a local MPC, which computes control inputs over a finite horizon to minimize tracking error and control effort. In the dynamic environment, the controller must follow the path while also avoiding the dynamic obstacles by changing its constraints in real time.

The quadcopter utilized in this project was the Crazyflie 2.0 platform. The robot was simulated using the existing PyBullet framework, which provided the body dynamics and a physics-based environment for control and motion planning development [3].

The quadcopter operates in a three-dimensional Euclidean workspace $W = \mathbb{R}^3$ with its configuration space defined as $SE(3)$.

Within simulation, the actuation input consists of the four motor rotational speeds, expressed in rotations per minute (RPM), defined as:

$$\mathbf{u} = [\omega_1, \omega_2, \omega_3, \omega_4]^T \tag{1}$$

For control purposes, the quadcopter is described using a 12-dimensional state vector,

$$\mathbf{x} = [x, y, z,\ \phi, \theta, \psi,\ \dot{x}, \dot{y}, \dot{z},\ \dot{\phi}, \dot{\theta}, \dot{\psi}]^T \tag{2}$$

where x, y, z denote position, ϕ, θ, ψ denote roll, pitch, and yaw, and the remaining states represent their respective velocities.

[1]The project outline, structure, and concepts of the solution were a collaborative work with all members of the group. The local MPC solver was implemented by Sam Leveau, which Enan Bester expanded into a dynamic object solver. The global RRT solver was designed by Enan Bester, which was expanded to other optimal algorithms such as RRT* and BIT* by Kai Henriquez. The definition of the environment and the porting to the simulation were done by Kai Henriquez. Hunor Kovacs implemented the performance metrics and organized the documentation, to which everybody contributed, especially their own area of expertise.

[2]Generative AI was used as a support tool for debugging all code and assisting with the creation of URDF files. The final implementations and results were produced by the authors, with all generated code being reviewed, modified when necessary, and verified by the authors.

[3]This project built upon the existing PyBullet code base: https://github.com/utiasDSL/gym-pybullet-drones. The state-space representation, global path planner, and MPC-based trajectory tracking controller were implemented by the authors.

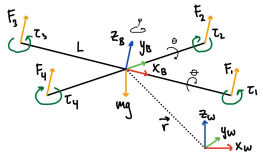
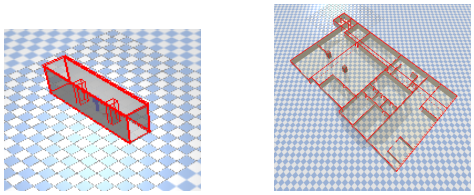


Fig. 1: Quadcopter body frame, world frame, thrust forces, and torque Directions

To represent a floor plan for a real-world building, two environments were created and utilized throughout the project. The first environment (Figure 2a) consists of a simple hallway with two pillar obstacles, in which two additional dynamic obstacles can be enabled. The second features a large floor plan of an office space (Figure 2b). Within these environments, global path planning was performed in the configuration space $\mathbb{R}^3$, corresponding to the position x, y, z.



(a) Hallway (b) Floor Plan

Fig. 2: Isometric views of tested environments

The following assumptions apply throughout this project:

- The environment and all static obstacles are fully known. Dynamic obstacles are unknown to the quadcopter.
- The drone is assumed to be differentially flat and is modeled around a nominal hover operating point.
- Full state feedback is available to the controller.
- External disturbances, such as wind and heat, are neglected.

III. MOTION PLANNING METHOD

A. Architecture Overview

The motion planning method consisted of two main components: a global planner and a local trajectory controller. First, a global planner is used to generate a collision-free path between start and end positions. This path is interpolated and provided to an MPC controller for trajectory tracking. At each timestep, the controller optimizes a local trajectory over a finite prediction horizon while respecting the drone's dynamics and operational constraints. An extended MPC formulation for dynamic obstacle handling was also implemented within the hallway environment.

These components are discussed in further detail below.

B. Global Path Planner

For the global planner, multiple algorithms were evaluated; each of them functions as follows:

- Rapidly-exploring Random Trees (RRT) incrementally builds a tree by randomly sampling states of the configuration space and attempting to connect each one through the nearest node by maximum step size. Collision checking is only performed along the expanded edges. This algorithm is probabilistically complete but does not guarantee optimal paths.
- Optimal Rapidly-exploring Random Tree (RRT*) builds upon RRT by improving the path quality. It rewires nearby nodes when a new sample is added to reduce the overall path cost. This allows the algorithm to converge on an optimal solution.
- RRT-connect is a bidirectional variant of RRT that simultaneously grows a tree from both the start and goal positions. At each successful addition of a sample to a tree, the other tree aggressively attempts to extend towards that sample to allow the trees to connect. This should allow the algorithm to converge on a solution faster, although not optimally.
- Batch Informed Trees (BIT*) is a sampling-based planner that incorporates ideas from graph-search algorithms without the need for discretization. It generates batches of samples and orders potential nodes according to a heuristic. It then incrementally builds an implicit Random Geometric Graph and solves in order of increasing estimated cost. After a solution is found and improved upon, the sampling space is bounded by an informed subset of the state space determined by the most optimal solution.

C. Static Local Planner

MPC and system dynamics are in discrete time indexed by $k \in N$. Let N denote the MPC prediction horizon. At each control step, MPC is provided a finite reference trajectory by the global path planner. The reference trajectory specifies desired position states, while all remaining state references are set to zero. It then optimizes a sequence of control inputs over the horizon, of which only the first control input is used. The control input u_k consists of the quadcopter's total thrust T and attitude torques τ_x, τ_y, τ_z corresponding to the roll, pitch, and yaw. These are then transformed into RPM values for use as inputs in the PyBullet simulation.

A continuous-time state space model was first derived by linearizing the quadcopter equations of motion around a nominal hover operating point under small-angle assumptions utilizing Figure 1. This continuous-time model was then discretized using a forward Euler approximation with sampling time T_s.

The MPC objective is to minimize the deviation from the reference trajectory while penalizing control effort. The finite-horizon cost function is defined as

$$J = \sum_{k=0}^{N-1} \left[(\mathbf{x}_k - \mathbf{x}_k^{\text{ref}})^\top \mathbf{Q} (\mathbf{x}_k - \mathbf{x}_k^{\text{ref}}) + \mathbf{u}_k^\top \mathbf{R} \mathbf{u}_k \right] + (\mathbf{x}_N - \mathbf{x}_N^{\text{ref}})^\top \mathbf{P} (\mathbf{x}_N - \mathbf{x}_N^{\text{ref}}) \tag{3}$$

where $\mathbf{Q}$ and $\mathbf{R}$ are state and input weighting matrices respectively, $\mathbf{P}$ is the terminal cost weight, $\mathbf{x}_n$ denotes the

terminal position, and $\mathbf{x}_N^{\text{ref}}$ denotes the terminal reference state. The terminal cost matrix $\mathbf{P}$ was computed by solving the Discrete-time Algebraic Riccati Equation (DARE).

The optimization is constrained by the initial condition, defined as the interpolated waypoint, and by the system dynamics. Hard attitude constraints were also incorporated by limiting the roll and pitch angles to $\pm\alpha$, where $\alpha = 30°$.

To improve feasibility and closed-loop stability, a terminal cost (already included in (3)) and a terminal set constraint were added. The terminal set is defined as a box constraint on the last position in the reference path,

$$\left|\mathbf{x}_{N,0:2} - \mathbf{x}_{N,0:2}^{\text{ref}}\right| \leq \varepsilon_{\text{pos}}, \tag{4}$$

where $\mathbf{x}_{N,0:2}$ denotes the terminal position components (x, y, z) and ε_{pos} represents the box constraint in meters, with $\varepsilon_{\text{pos}} = 0.75$.

The quadratic program was solved using the CVXPY optimization framework with the OSQP solver.

D. Dynamic Local Planner

To approximate real-world operation, an extension is made to the static MPC formulation to account for dynamic obstacles. The drone is assumed to have access to onboard perception and state estimation capabilities, including point cloud imaging, object detection via clustering, and prediction of future obstacle states through Kalman filtering and/or constant velocity prediction. Emulating this within simulation is done via the random sampling of points on each object's surface and associating each point with an object ID i. The positions of moving objects are also known ahead of time to give a "prediction" of obstacle state.

Working within the limitations of the CVXPY solver, the closest point of each obstacle with respect to the drone is found. Each point is then used to generate a half-space constraint representing the respective object such that

$$\mathbf{n}_i \left(\mathbf{p}_{drone} - \mathbf{p}_{close,i}\right) \geq D \tag{5}$$

with

$$\mathbf{n}_i = \frac{\mathbf{p}_{drone} - \mathbf{p}_{close,i}}{||\mathbf{p}_{drone} - \mathbf{p}_{close,i}||} \tag{6}$$

Where $\mathbf{p}_{drone}$ is the current drone position, $\mathbf{p}_{close,i}$ is the closest point of object i to the drone, and D is a chosen safety distance from the object.

For each step of the MPC horizon ($k = 0, \ldots, N$), $\mathbf{p}_{near,i}$ is recomputed based on the current drone position ($k = 0$) and the predicted position of the object at horizon step k. This allows the half-space constraints to evolve around each obstacle as it moves.

The chosen method for dynamic obstacle avoidance has several limitations, making it suboptimal for practical applications. The first limitation is introduced by the choice of solver. Although fast, CVXPY only allows convex optimization problems; thus, only half-space constraints are allowed. The result of this is failure to find a solution in several configurations. Although soft constraints could mitigate this issue, the operational scenario may require navigating around people; thus, hard constraints are preferred. Furthermore, recomputing the half-space constraints for each time step slows down the solver significantly.

IV. RESULTS

A. Performance metrics

Our performance metrics focused on the operation of the MPC algorithm given a globally planned reference path. The primary metric is the *root mean squared error* (RMSE) between the planned global path and the executed local path. By evaluating this metric, the accuracy of the path tracking can be obtained, allowing for comparison and tuning of the MPC implementation. Other performance metrics taken are the *average velocity*, which is calculated using the *path length,* and the *completion time*. Additionally, the failed trials were also recorded and used to calculate the *success rate* metric.

B. Description of results

1) Environment comparison: The performance metrics were evaluated in both environments using BIT*. The hallway and floor plan were tested 15 and 11 times respectively as seen in Table I.

TABLE I: Results

	Hallway	Floor plan
Success rate	100%	91%
RMS error [mm]	31.700	55.673
Average velocity [m/s]	0.649	0.991
Completion time [s]	12.9	48.425
Path length [m]	8.486	48.02

The key differences observed are the higher tracker error and increased average velocity in the floor plan environment. This result can be attributed to the tighter space in the hallway environment. As the distances increase with the floor plan environment the drone tends to move faster which results in higher errors when performing turns.

2) Local planner comparison: To evaluate the effect of MPC tuning,[4] Three different weight configurations were tested in the hallway environment, each with 10 trials. Specifically different configurations of the state weight matrix, $\mathbf{Q}$, and input weight matrix $\mathbf{R}$, were tested while keeping the remaining finite MPC horizon ($N = 20$), and trajectory interpolation the same ($n_{points} = 200$). The resulting RMS position error is plotted over the average velocity for each MPC tuning. These results are seen Table II Figure 3, with each set of weight matrices corresponding to the cyan, purple, and orange results, respectively.

MPC version 1:

$$\mathbf{Q} = \begin{bmatrix} 20 & 20 & 10 \\ 10 & 10 & 0.001 \\ 3 & 3 & 3 \\ 1.5 & 1.5 & 0.001 \end{bmatrix} \quad \mathbf{R} = \begin{bmatrix} 0.1 & 2 & 2 & 0.01 \end{bmatrix}$$

[4]Due to the limitations discussed in section III-D and a focus on the general MPC method, no dynamic MPC results were recorded.

This configuration assigns moderate weights to position and velocity states while adding a larger penalty to the control inputs. As a result, the controller displays smooth behavior with limited aggression in correcting the deviations.

MPC version 2:

$$\mathbf{Q} = \begin{bmatrix} 50 & 50 & 10 \\ 5 & 5 & 0.001 \\ 1 & 1 & 3 \\ 1.5 & 1.5 & 0.001 \end{bmatrix} \quad \mathbf{R} = \begin{bmatrix} 0.1 & 0.5 & 0.5 & 0.01 \end{bmatrix}$$

In this set of tuning matrices, higher weights were assigned to the position states while the input penalties were reduced. This made the controller apply stronger actions, leading to more aggressive behavior. As a consequence, the tracking accuracy was improved and higher average velocities were observed. However, the increased aggression also occasionally resulted in failed attempts due to overshoot.

MPC version 3:

$$\mathbf{Q} = \begin{bmatrix} 10 & 10 & 5 \\ 3 & 3 & 0.001 \\ 3 & 3 & 3 \\ 1.5 & 1.5 & 0.001 \end{bmatrix} \quad \mathbf{R} = \begin{bmatrix} 0.1 & 0.1 & 0.1 & 0.01 \end{bmatrix}$$

In contrast to the previous version, this configuration lowers the weights on both the position states and control inputs. This controller demonstrated weaker correction, which led to slower movement and higher tracking error.

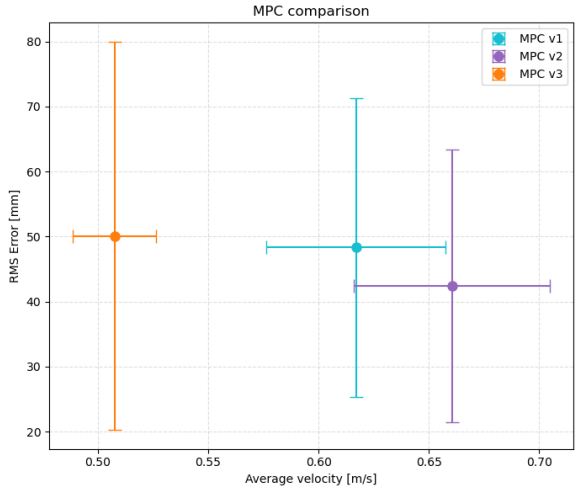


Fig. 3: Average Velocity & RMS tracking error for multiple MPC weight matrices

TABLE II: MPC Results

	MPC V1	MPC V2	MPC V3
Success rate	96%	73%	94%
RMS error [mm]	48.32	42.42	50.11
Average velocity [m/s]	0.62	0.66	0.51
Completion time [s]	13.96	13	16.97
Path length [m]	8.66	8.58	8.65

These results show that the second tuning achieved the best performance. However, the MPC controller was too aggressive and failed in multiple attempts. For a reliable and quick result, the recommended MPC tuning is MPC Version 1 (cyan).

3) Global path planner comparison: Within the hallway environment, a comparison of global path planners was also performed, with 10 trials performed for each algorithm. The computation time and path optimality, dictated by path length, were then compared (Figure 4).

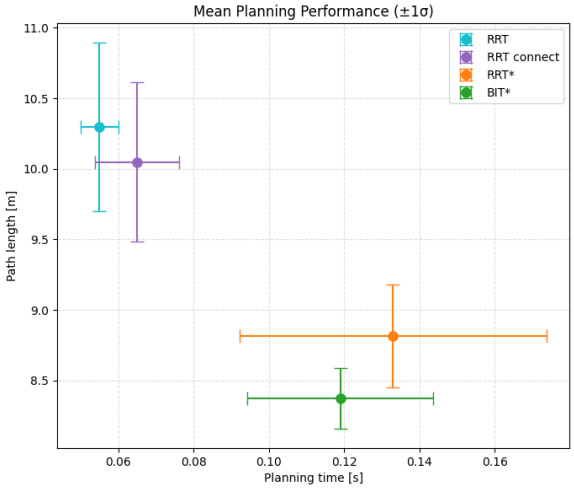


Fig. 4: Time taken & length of generated global path

The results show that RRT and RRT-connect perform the fastest but have a significantly longer path length. In comparison, the algorithms including optimization (RRT* and BIT*) took longer but with a more optimal path. For this application, BIT* showed the best performance, managing the lowest path length and a lower computation time than RRT* due to being bounded by an informed subspace.

V. DISCUSSION AND CONCLUSION

A. Theoretical discussion

1) Optimality: The global path planning methods that were tested vary in optimality; RRT and RRT-connect have no manner of optimizing the generated path length, resulting in longer path lengths (Figure 4) when tested against RRT* and BIT*, which are asymptotically optimal.

By design, both the MPC and dynamic extension compute an optimal control sequence at each control step. The use of a terminal set and cost enables recursive feasibility and stability.

2) Completeness: All of the global planners used are probabilistically complete. While there isn't a way to make them deterministically complete, they can drastically improve the results by performing informed sampling or using heuristics, as seen with the BIT* algorithm. Ideally, implementing these extensions would allow the algorithm to find a workable path faster, making it more probabilistically complete.

3) Complexity: The complexity of the environment has a significant effect on the ability of the global planner to successfully generate a path for it. For example, the BIT* algorithm was the only one that could reliably produce a result for the floor plan environment. This is partially due to the fact that BIT* makes use of heuristics and informed sampling, allowing it to find solutions much more efficiently. RRT, RRT* and RRT-connect were not able to solve the environments within their allotted sample limit, and while adjusting their specifications could improve the success rate, it makes them less generalizable.

The computational complexity of the developed MPC planner was increased by the optimization solved at each control step and scaled with the number of states, the prediction horizon length, and the constraints. In particular, obtaining a real-time feasible MPC solution became increasingly difficult as the prediction horizon was increased. Additionally, extending the MPC formulation to dynamic environments further increased computational complexity, resulting in computation times that were unrealistic.

B. Practical discussion

1) Performance: Overall, BIT* was the best-performing algorithm despite being more complex. It consistently produced the shortest path for the hallway environment and was the only one that could reliably produce a result in the floor plan environment. While this does come with an increased compute time, this should be permissible for the intended use case.

As demonstrated in Table I, the MPC controller generally demonstrated strong performance tracking by achieving a low average RMS error and completing the task in reasonable time. It was also found that higher velocities were associated with more aggressive control actions, which reduced tracking error but increased the chance of overshoot and instability.

2) Alternatives: While other sampling-based planners may outperform BIT* under specific situations, no sampling-based planner is strictly superior. Further performance gains are more likely to be achieved with trajectory optimization rather than replacing the planner.

At the local planner level, a sampling-based controller such as Model Predictive Path Integral (MPPI) could have improved handling of nonlinear dynamics and the dynamic obstacles, as it directly optimizes control sequences through rollouts without requiring linearization of the system dynamics. In turn, MPPI could theoretically increase average velocity and reduce RMS error, but at a significantly higher computational cost.

3) Limitations: Currently the global solver does not account for the dynamics of the quadcopter, which may lead to dynamically infeasible paths during movements such as sharp turns that the vehicle cannot perform. Furthermore, for a truly real-world application, the environment likely won't be known in its entirety, and the local planner would have to operate on a path generated by an online planner being fed a limited view. The current global planners used require a full representation of the environment to determine a feasible path and are not run repeatedly.

From a computational perspective, the MPC prediction horizon was constrained by the authors' computers; particularly with the implementation of the dynamic extended controller. Increasing the horizon length also significantly increased the solver run times.

C. Improvements

As an initial step, improvements could be made by revisiting the assumptions that were proposed in the problem definition. For this, measurement uncertainty could be incorporated into the modeling of objects. Additionally, the assumption of differential flatness could be removed, solving the drone model for roll and pitch, allowing higher-speed maneuvers. The model could also include wind and temperature changes in the environment.

Furthermore, despite the effectiveness of BIT*, there are additional steps that could be taken to assist it in converging on a solution faster. An observation of the RRT-connect algorithm was how it was able to steer its trees towards each other, even if ultimately failing to consistently provide solutions. A bidirectional version of BIT* with a connect function that allows the 2 trees to pull each other together may result in even faster solutions, although managing 2 tree queues may prove difficult.

Lastly, the additional dynamic MPC scheme would benefit from a non-convex solver. This would allow for simpler and more accurate constraint formulation around moving obstacles. This would decrease the likelihood of non-feasible solutions and may reduce computation time relative to the current approach, allowing feasible real-time computation.

D. Conclusion

In conclusion, the results of this project demonstrate that the use of BIT*, coupled with a well-tuned MPC controller and known environment information, enables the first step for emergency indoor quadcopter navigation.

References

[1] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Batch Informed Trees (BIT*)," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2015, pp. 3067–3074

[2] J. J. Kuffner and S. M. Lavalle, "RRT-connect: An Effficient Approach to single-query path planning," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2000, pp. 995–1001

[3] A. Restas, "Unmanned Aerial Vehicles for Search and Rescue," *International Journal of Emergency Management*, vol. 12, no. 3, pp. 231–245, 2015.

[4] Y. Zhang, L. Liu, and X. Wang, "A Survey of Indoor UAV Obstacle Avoidance Research," *Drones*, vol. 7, no. 2, 2023.

[5] M. Bangura and R. Mahony, "Real-Time Model Predictive Control for Quadrotors," in *Proceedings of the IFAC World Congress*, 2014, pp. 11773–11780.

[6] C. Zammit and E.-J. van Kampen, "Comparison of A* and RRT in Real-Time 3D Path Planning of UAVs," *Proceedings of the AIAA Scitech 2020 Forum*, Orlando, FL, USA, 2020, AIAA Paper 2020–0861. doi: 10.2514/6.2020-0861.

[7] S. Leveau, K. Henriquez, E. Bester, and H. Kovacs, "Ember_PDM," GitHub repository, 2026. [Online]. Available: `https://github.com/samuroo/Ember_PDM`